UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Ingeniería en Sistemas de la Información

CONNECTA: CHAT EMPRESARIAL

Programación orientada a objetos

Integrantes:

Camila Lascano

Ashley Espinoza

Paula Simbaña

CONNECTA- Sistema de Chat Empresarial Distribuido

1. Descripcion de nuestro proyecto

Este informe presenta el prototipo CONNECTA, un chat empresarial desarrollado en Go como parte del segundo entregable del Proyecto Integrador de tercer semestre. El documento organiza los avances técnicos, las pruebas realizadas y las evidencias de trabajo, procurando distinguir las funcionalidades comprobadas de los componentes pendientes.

Integrante	Responsabilidad
Paula Gabriela Simbaña Pérez	Persona 1: servidor Go y rutas HTTP
Nadia Camila Lascano Alemán	Persona 2: cliente Go y menú de terminal
Ashley Michelle Espinoza Rosero	Persona 3: almacenamiento JSON, integración y documentación

El informe y el video corresponden a actividades compartidas. Las contribuciones registradas mediante commits se describen solo cuando existe evidencia de su autoría. El número de figura, su título y la nota explicativa aparecen en el mismo orden que exige APA 7.

2. Nombre del sistema seleccionado

El sistema seleccionado se denomina CONNECTA: Chat Empresarial. La opción escogida corresponde al desarrollo de un chat empresarial cliente-servidor. El prototipo permite que usuarios registrados intercambien mensajes de texto desde una aplicación de terminal y consulten el historial de conversaciones.

El cliente y el servidor están desarrollados en Go. El intercambio de información se realiza mediante solicitudes HTTP y respuestas JSON. Para conservar los registros después de reiniciar el servidor se incorporó un módulo de persistencia en archivos JSON, sin instalar una base de datos externa.

3. Objetivo del prototipo

El objetivo es desarrollar y comprobar un prototipo funcional de comunicación entre usuarios, con un cliente y un servidor independientes, usando Go y conceptos vistos en el semestre. La aplicación permite registrar personas, gestionar sesiones básicas, enviar mensajes, consultar usuarios y recuperar el historial de una conversación.

Como parte de la responsabilidad de almacenamiento, se buscó corregir la pérdida de información que ocurría cuando el servidor terminaba. Para ello se agregaron métodos de lectura, escritura, conversión y reconstrucción de registros JSON. Al finalizar las pruebas, se verificó que usuarios y mensajes volvían a aparecer después de reiniciar CONNECTA.

4. Funcionalidades implementadas

El prototipo cuenta con una parte interactiva y otra de procesamiento en el servidor. Las operaciones del menú están conectadas con rutas HTTP que reciben y devuelven información. Los mensajes se almacenan con el ID del emisor, el destinatario, el contenido y una fecha de envío.

Funcionalidad	**Resultado documentado**
Registro y listado de usuarios	Comprobados mediante solicitudes HTTP y la respuesta del cliente.

Inicio y cierre de sesión	Comprobados desde el menú del cliente Go, utilizando ID.
Envío de mensajes	Comprobado mediante curl y desde el cliente Go.
Consulta de historial	Se recuperaron dos mensajes entre los usuarios 1 y 2.
Persistencia de usuarios	Recuperación de Ashley y Paula después del reinicio.
Persistencia de mensajes	Recuperación del mensaje con ID 1 después del reinicio.
Control de errores en almacenamiento	Pruebas de archivo inexistente, JSON inválido y registros incorrectos.
Pruebas y compilación	go build ./... y go test ./... sin errores en las capturas.

Las operaciones anteriores son las que se observaron en las evidencias. No se afirma que exista una interfaz gráfica, cifrado TLS o una autenticación con contraseña, porque esos elementos no forman parte del prototipo probado.

5. Requisitos atendidos y pendientes

Para valorar el alcance del segundo entregable, se relacionaron los requisitos principales con las evidencias disponibles. Esta revisión permite diferenciar lo que ya funciona de lo que corresponde a una etapa posterior.

Requisito	Estado	Sustento
Servidor Go y rutas HTTP	Atendido	Arranque y GET /salud, Figura 20.
Cliente Go con menú	Atendido	Inicio de sesión y envío, Figura 32.
Registro, listado e historial	Atendido	Figuras 21, 22, 29 y 33.
Almacenamiento y recuperación JSON	Atendido	Figuras 22, 23, 29 y 30.
Manejo de errores	Atendido, básico	Pruebas del paquete storage, Figuras 7 y 13.
Autenticación con contraseña	Pendiente	La sesión actual utiliza únicamente ID.
Servicio FastAPI integrado	Pendiente	La carpeta existe, no hay servicio funcional integrado.
Base de datos relacional	Pendiente / fuera de esta fase	Se utilizan archivos JSON.
Video demostrativo	Pendiente de enlace	No se recibió un enlace del video.
README principal consolidado	Por verificar	Existen README en raíz y en app-go; revisar la versión final.

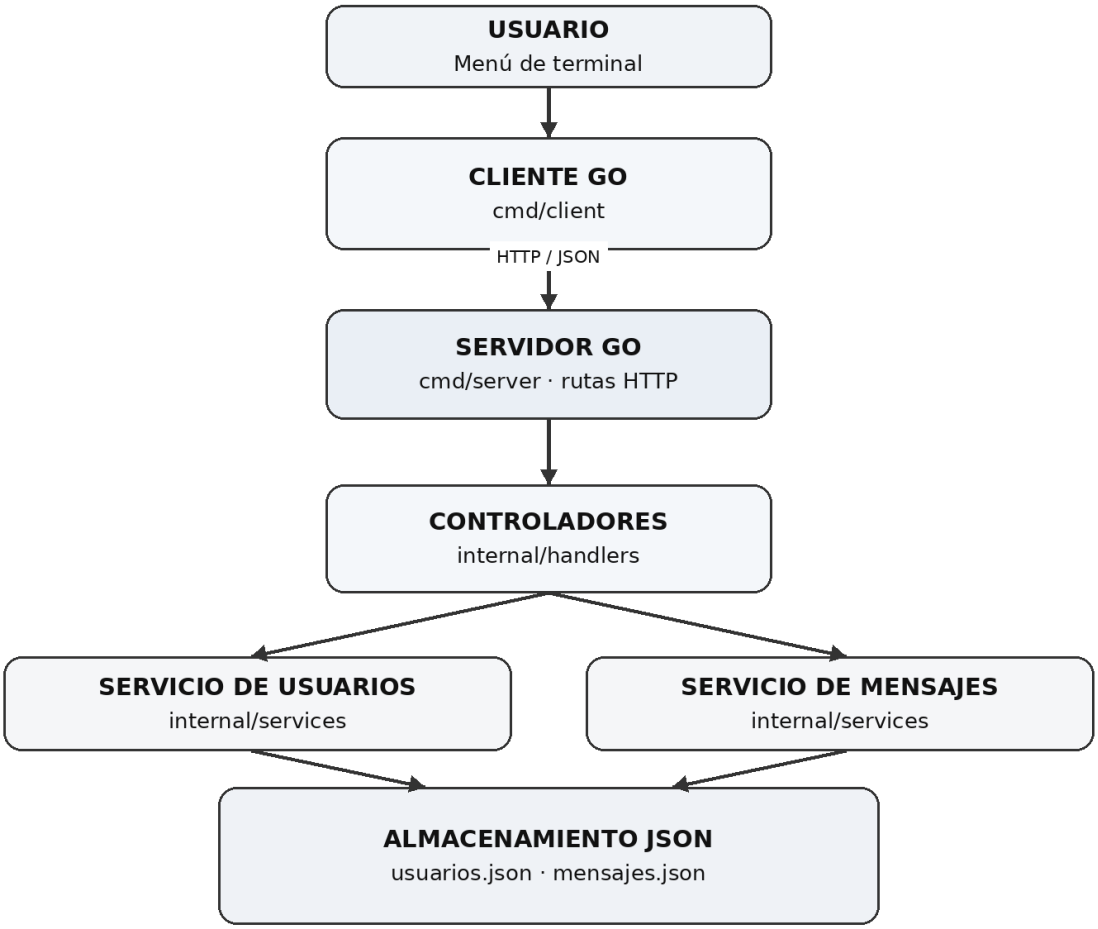
La persistencia de archivos JSON fue incorporada como mejora funcional. El sistema continúa siendo un prototipo académico y no se considera listo para una empresa real, especialmente por el alcance limitado de la seguridad y las sesiones.

6. Diagrama actualizado de componentes, módulos o estructuras

La arquitectura actual está dividida en cuatro niveles: el cliente Go, el servidor y sus controladores HTTP, los servicios que contienen la lógica de usuarios y mensajes, y el módulo de archivos JSON. La Figura 1 representa esta organización a partir de los archivos revisados.

Figura 1

Arquitectura funcional del prototipo CONNECTA



Nota. Las solicitudes salen del menú del cliente Go y llegan al servidor mediante HTTP. Los controladores llaman a los servicios, que utilizan internal/storage para guardar o recuperar usuarios y mensajes en archivos JSON. El servicio FastAPI previsto no forma parte del flujo implementado. Diagrama de elaboración propia a partir del código de CONNECTA.

Dentro de app-go/cmd/client se encuentra la lógica del menú y las solicitudes. app-go/cmd/server contiene la configuración del servidor y las rutas. Los paquetes internal/handlers, internal/models, internal/services e internal/storage permiten separar responsabilidades sin duplicar las funciones existentes.

7. Explicación de la comunicación cliente-servidor

CONNECTA utiliza el modelo de solicitud y respuesta. Primero el usuario selecciona una opción en el cliente Go. El cliente construye una petición HTTP, incluye los datos necesarios y la envía a la dirección del servidor. El servidor recibe la petición mediante una ruta, llama al servicio correspondiente y prepara una respuesta JSON. Luego el cliente interpreta esa respuesta y muestra el resultado en la terminal.

7.1. Operaciones y rutas

Método	**Ruta**	**Propósito**
GET	/salud	Consultar el estado del servidor
POST	/usuarios/registrar	Crear usuario
GET	/usuarios	Listar usuarios
GET	/usuarios/buscar	Buscar un usuario por ID
POST	/usuarios/iniciar-sesion	Iniciar sesión básica
POST	/usuarios/cerrar-sesion	Cerrar sesión
POST	/mensajes/enviar	Enviar y almacenar un mensaje
GET	/mensajes/recibir	Consultar mensajes recibidos
GET	/mensajes/historial	Consultar mensajes entre dos usuarios

7.2. Ejemplo de un envío

En la prueba, Ashley inició sesión desde el cliente con el ID 1 y envió un mensaje a Paula, con ID 2. El cliente realizó una solicitud al endpoint /mensajes/enviar. El servidor validó los usuarios, creó el registro y lo incorporó al archivo mensajes.json mediante el servicio persistente. Al consultar la opción de historial, el cliente recuperó tanto el mensaje anterior de prueba como el nuevo envío, con sus fechas.

Las pruebas se realizaron en el mismo entorno, con el servidor en 127.0.0.1 y el puerto 8080. Esta configuración local es suficiente para demostrar la comunicación entre dos procesos, aunque no equivale a una publicación accesible en Internet.

8. Descripción de estructuras, constructores y métodos

8.1. Modelos utilizados

Los modelos ya existentes representan los usuarios y mensajes del sistema. Usuario contiene ID, nombre y estado de sesión. Mensaje reúne ID, remitente, destinatario, contenido y fecha. Estos modelos utilizan campos privados y métodos para consultar información, por lo que el módulo de persistencia no intenta serializarlos directamente.

Estructura	Elementos principales	Uso
models.Usuario	id, nombre, activo	Identifica cada usuario y su estado de sesión.
models.Mensaje	id, remitenteID, destinatarioID, contenido, fecha	Representa un mensaje de una conversación.
storage.UsuarioGuardado	ID, Nombre	Registro serializable a

		JSON.
storage.MensajeGuardado	ID, RemitenteID, DestinatarioID, Contenido, Fecha	Registro serializable con fecha original.
storage.ArchivoJSON	ruta	Guarda y lee el archivo configurado.
services.UsuarioService	map, siguiente, mutex, archivo	Administra usuarios y su persistencia.
services.ChatService	usuarios, mensajes, siguiente, mutex, archivo	Administra mensajes y su persistencia.

8.2. Funciones de la Persona 3

Función o método	**Función explicada**
NuevoArchivoJSON(ruta)	Crea una estructura que conoce la ruta del archivo.
ArchivoJSON.Guardar(datos)	Convierte los datos a JSON, crea directorio y escribe mediante archivo temporal.
ArchivoJSON.Cargar(destino)	Lee JSON y recupera el contenido, controlando ausencia o error de lectura.
ConvertirUsuarios / ConvertirMensajes	Prepara información de los modelos originales para serialización.
ReconstruirUsuarios / ReconstruirMensajes	Recupera objetos y valida identificadores y

	datos.
NewMensajeConFecha	Reutiliza el constructor existente y conserva la fecha original.
NewUsuarioServiceConArchivo	Recupera usuarios y restablece el siguiente ID disponible.
NewChatServiceConArchivo	Recupera mensajes, valida usuarios relacionados y restablece el siguiente ID.
guardarUsuariosBloqueados / guardarMensajesBloqueados	Persisten los registros durante las operaciones protegidas con mutex.

8.3. Manejo de errores y decisiones

El método Guardar convierte los datos mediante json.MarshalIndent y crea un archivo temporal antes de reemplazar el archivo de destino. Con esta medida se reduce el riesgo de dejar un archivo parcial si la escritura falla. El método Cargar no considera un error la ausencia del archivo en el primer inicio, pero sí devuelve errores cuando encuentra contenido JSON inválido.

Para la recuperación se comprueban identificadores duplicados y registros inválidos. En el servicio de chat también se verifica que existan el remitente y destinatario del mensaje recuperado. En el registro de usuarios y mensajes, la aplicación intenta revertir el cambio en memoria si ocurre un error al persistir. No se implementó un sistema transaccional de base de datos.

El trabajo aprovecha structs, constructores, métodos, receptores por puntero, encapsulamiento, paquetes, manejo de errores y lectura/escritura de archivos. No se añadieron

interfaces complejas porque la aplicación necesita únicamente un mecanismo sencillo de almacenamiento en esta fase.

9. Evidencias de ejecución y pruebas

Las evidencias provienen de capturas reales del desarrollo en GitHub Codespaces. En este apartado se describen los resultados principales; en el Anexo A se incorporan las 32 capturas cronológicas, cada una como figura numerada con una nota explicativa APA 7. Los resultados descritos se limitan a lo que se ve en las capturas.

9.1. Pruebas automáticas

Antes de implementar la persistencia se comprobó que el proyecto compilaba y ejecutaba sus pruebas originales. Después se crearon pruebas del paquete internal/storage para confirmar la lectura, escritura, recuperación y validación de datos. La Figura 13 muestra nueve pruebas aprobadas en el módulo de almacenamiento. La salida [no test files] de cliente, servidor y handlers significa que esas carpetas no contienen archivos de pruebas automatizadas, no que la compilación haya fallado.

9.2. Prueba de persistencia de usuarios

Se consultó inicialmente la lista de usuarios y apareció vacía. Posteriormente se registraron Ashley con ID 1 y Paula con ID 2. El contenido del archivo usuarios.json confirmó la escritura. Tras detener y reiniciar el servidor, una nueva consulta devolvió ambos registros con sus IDs originales y sesiones inactivas, como muestra la Figura 23. Luego se creó Camila con ID 3, verificando que el contador continuaba correctamente, como muestra la Figura 24.

9.3. Prueba de persistencia de mensajes

Se envió mediante curl un mensaje de Ashley a Paula y se comprobó que el archivo mensajes.json contenía el ID, los usuarios relacionados, el contenido y la fecha. La Figura 29 muestra el registro inicial. Después de reiniciar el servidor, la consulta del historial devolvió el mismo mensaje con su fecha original, como se observa en la Figura 30. Esto demuestra persistencia de ese mensaje durante la prueba realizada.

9.4. Prueba de integración del cliente

Se ejecutaron servidor y cliente Go como procesos independientes. Desde el cliente se inició sesión con Ashley, se envió otro mensaje a Paula, se consultó el historial y se cerró la sesión. En el historial aparecieron dos mensajes, ambos con sus fechas. Estas operaciones se observan en las Figuras 32 y 33.

Prueba operativa	Evidencia	Resultado registrado
Compilación y test inicial	Figura 3	Compilación correcta; pruebas preexistentes correctas.
Pruebas JSON	Figuras 7 y 13	Lectura, escritura y validaciones aprobadas.
Registro HTTP y JSON	Figura 22	Se guardaron Ashley y Paula.
Recuperación de usuarios	Figura 23	Reaparecieron tras reiniciar.
Continuidad de IDs	Figura 24	Camila recibió el ID 3.
Escritura de mensajes	Figura 29	Mensaje presente en

		mensajes.json y en historial.
Recuperación de mensajes	Figura 30	El mensaje y fecha reaparecieron tras reiniciar.
Cliente interactivo	Figuras 32 y 33	Inicio, envío, historial y cierre correctos.

10. Limitaciones actuales del prototipo

Aunque el prototipo cumple las operaciones esenciales probadas, existen limitaciones que conviene dejar claras en la entrega. La autenticación actual identifica al usuario por su ID y no comprueba contraseñas ni tokens. Las rutas HTTP no incluyen TLS, por lo que no se recomienda usar el sistema con datos reales de una empresa. Tampoco existen permisos por roles, mensajería grupal ni interfaz gráfica.

La información se guarda en archivos JSON locales. Esta solución funciona para demostrar persistencia, pero no sustituye una base de datos con transacciones, controles de acceso, copias de seguridad y escalabilidad. El módulo FastAPI anunciado en la estructura del repositorio no está integrado con el cliente y servidor utilizados en las pruebas. Los paquetes de cliente, servidor y controladores todavía carecen de pruebas automáticas específicas.

Al momento de preparar este informe, el video final no tiene un enlace confirmado. Además, el repositorio contiene un README en la raíz y otro en app-go: es necesario revisar que la información consolidada quede en el archivo solicitado para la entrega. Estas observaciones corresponden a requisitos pendientes y no se presentan como funcionalidades terminadas.

11. Enlace al repositorio Git

Repositorio del equipo: https://github.com/ashweeps/connecta-chat-empresarial

Rama de la Persona 3: https://github.com/ashweeps/connecta-chat-empresarial/tree/feature/almacenamiento-documentacion

La contribución de almacenamiento está registrada en tres commits: d85ebca (módulo JSON y pruebas), a36c102 (conversión y reconstrucción) y 2a1cbcf (integración de persistencia con el servidor). Las Figuras 8, 16 y 31 muestran la publicación de esas contribuciones. La integración definitiva a main depende de la revisión del equipo y del Pull Request correspondiente, por lo que no se afirma que ya esté completada.

12. Enlace al video demostrativo

Enlace al video: [Pendiente de colocar cuando el equipo publique la grabación].

La demostración debe presentar a las tres integrantes e incluir el servidor, el menú del cliente, el registro o consulta de usuarios, un envío de mensaje, la consulta del historial, el contenido de los archivos JSON y la recuperación después de reiniciar el servidor. El Anexo B ofrece un orden breve para explicar cada participación sin atribuir a una estudiante funciones realizadas por otra.

Referencias

American Psychological Association. (s. f.). Student paper checklist (Publication Manual): APA Style, 7th edition. https://apastyle.apa.org/instructional-aids/publication-manual-formatting-checklist.pdf

Equipo CONNECTA. (2026). CONNECTA: Chat empresarial [Código fuente]. GitHub. https://github.com/ashweeps/connecta-chat-empresarial

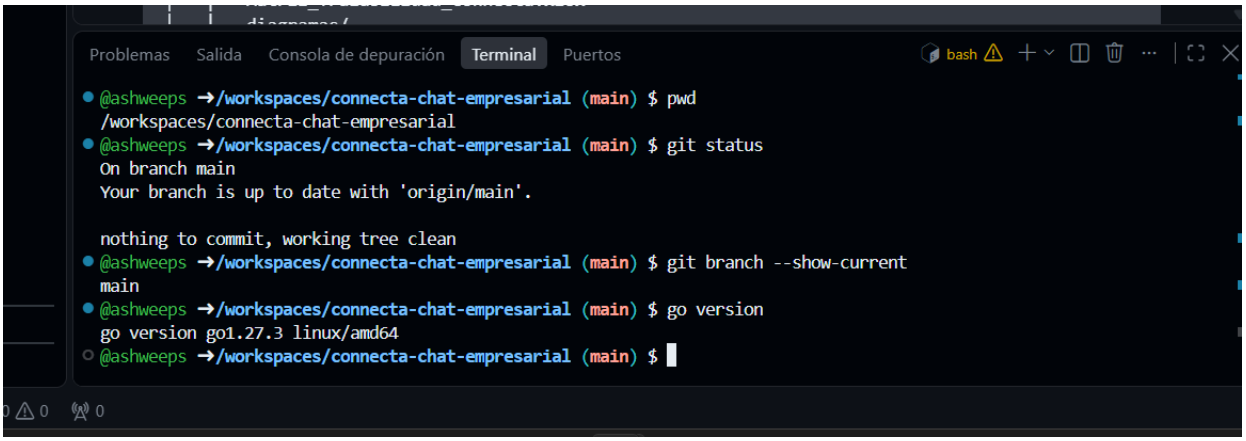
Purdue Online Writing Lab. (s. f.). APA tables and figures. Purdue University. https://owl.purdue.edu/owl/research_and_citation/apa_style/apa_formatting_and_style_guide/apa_tables_and_figures.html

Anexo A. Evidencias cronológicas con notas APA 7

El anexo reúne todas las capturas compartidas durante el desarrollo del almacenamiento y su integración con CONNECTA. Cada figura contiene una captura sin alterar, un título descriptivo y una nota que explica qué se observa y qué alcance tiene la evidencia. Las figuras se numeran de forma continua desde el diagrama de arquitectura.

Figura 2

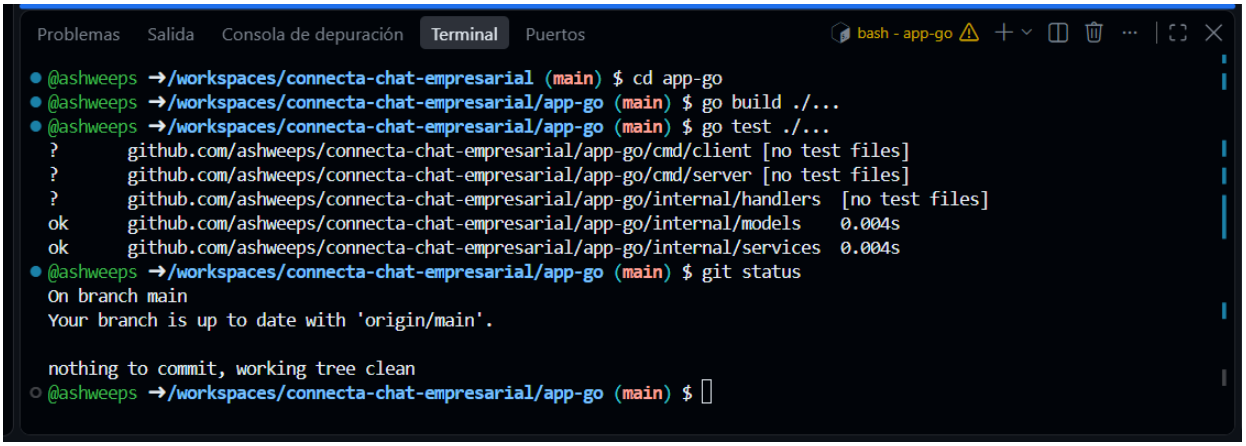
Verificación inicial del entorno en GitHub Codespaces



Nota. La captura muestra la carpeta del proyecto, la rama principal, el estado sin cambios pendientes y la versión Go 1.27.3. Se utiliza para identificar las condiciones de trabajo antes de desarrollar el almacenamiento. Captura de elaboración propia.

Figura 3

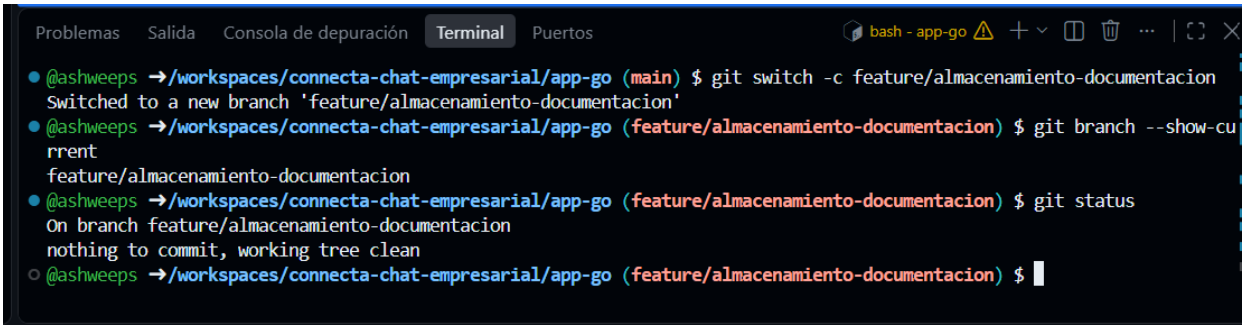
Compilación y pruebas del prototipo antes de los cambios



Nota. La terminal muestra go build ./... sin errores y resultados correctos en las pruebas existentes de modelos y servicios. La revisión permitió comprobar el estado inicial del proyecto antes de modificar archivos. Captura de elaboración propia.

Figura 4

Creación de una rama para la contribución de almacenamiento



Nota. Se observa la creación de la rama feature/almacenamiento-documentacion y la confirmación de que está activa. Esta separación permitió trabajar sin modificar directamente la rama principal. Captura de elaboración propia.

Figura 5

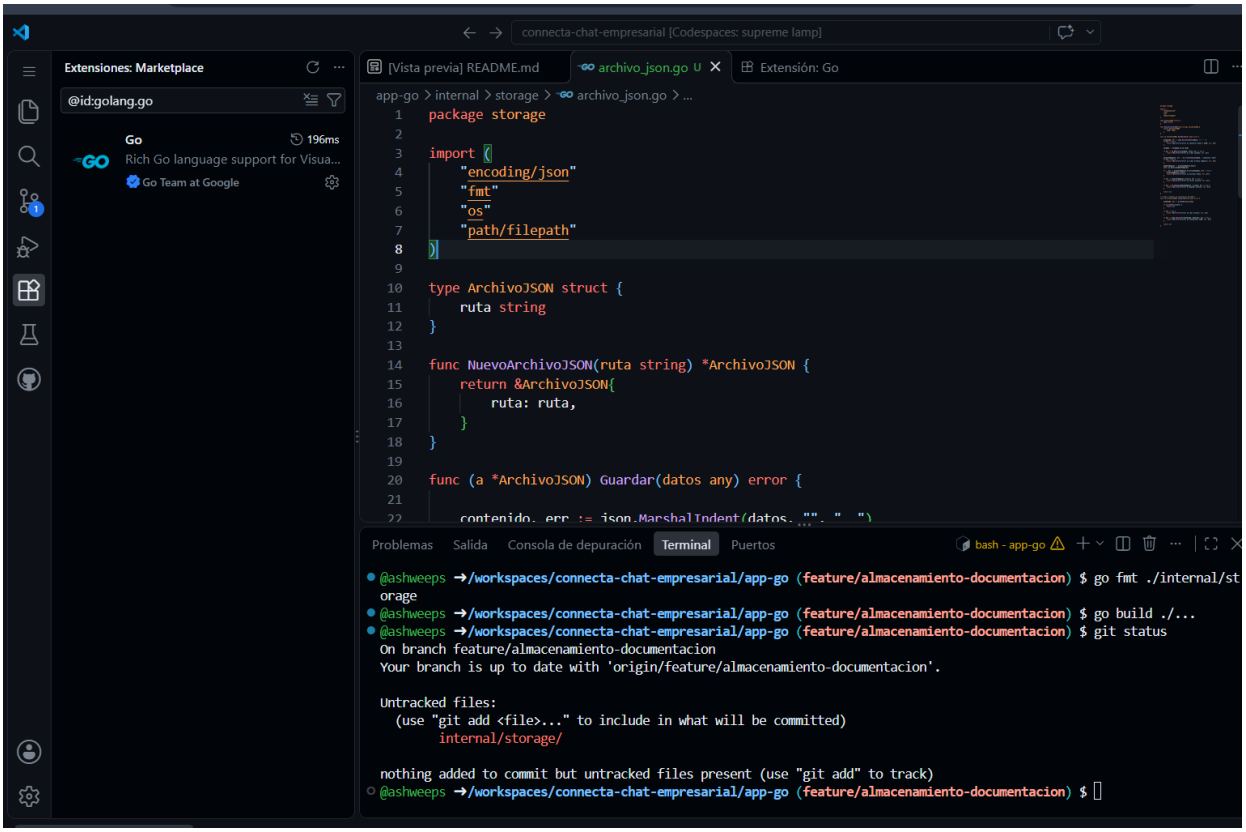
Publicación de la rama de trabajo en GitHub



Nota. La salida de git push confirma que la rama de almacenamiento se publicó en el repositorio remoto. La vinculación con origin permite enviar después los commits de la Persona 3. Captura de elaboración propia.

Figura 6

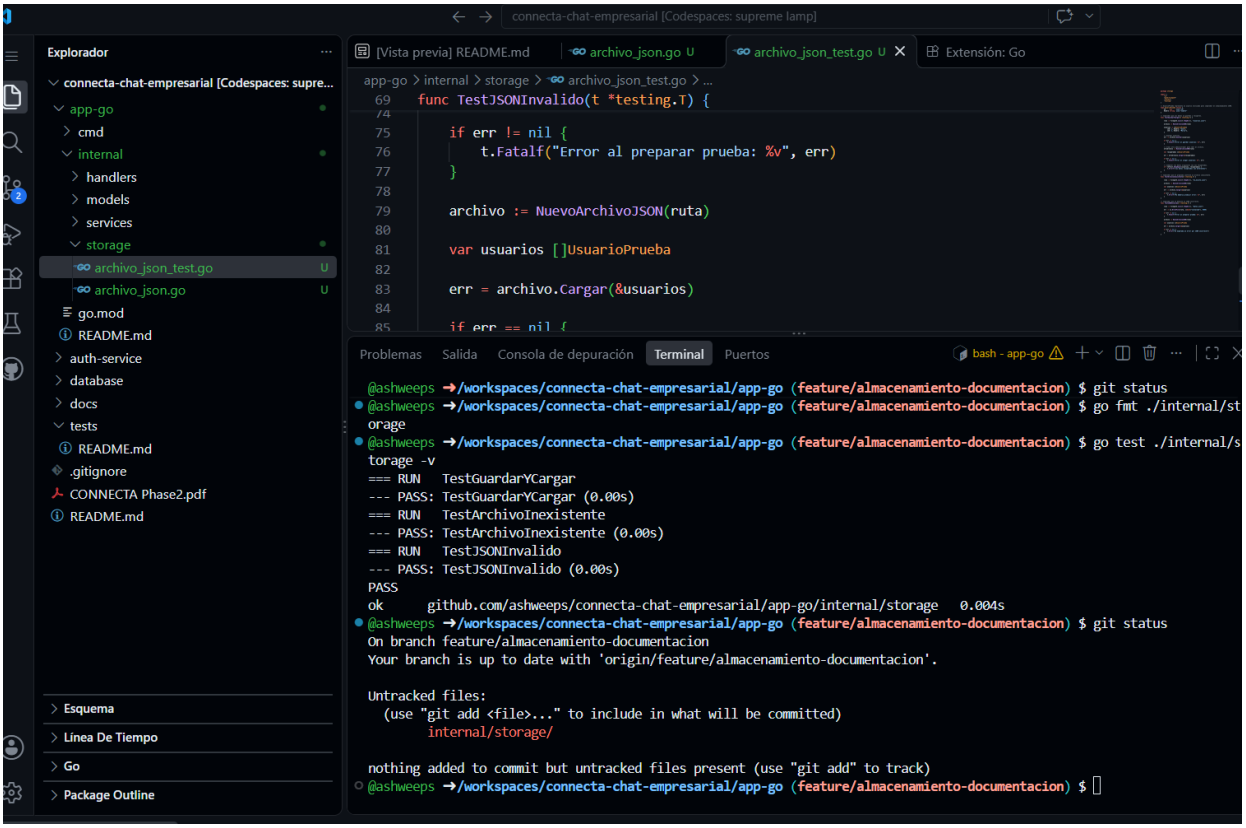
Creación del archivo de manejo JSON en Go



Nota. Se muestra el archivo archivo_json.go dentro de internal/storage y la compilación sin errores. El módulo contiene una estructura y métodos para gestionar la escritura y lectura de archivos JSON. Captura de elaboración propia.

Figura 7

Pruebas iniciales de lectura, escritura y errores de JSON



Nota. Se observan tres pruebas automatizadas aprobadas: guardar y cargar información, gestionar un archivo inexistente y detectar contenido JSON incorrecto. Estas pruebas corresponden al módulo aislado. Captura de elaboración propia.

Figura 8

Primer commit del módulo de almacenamiento

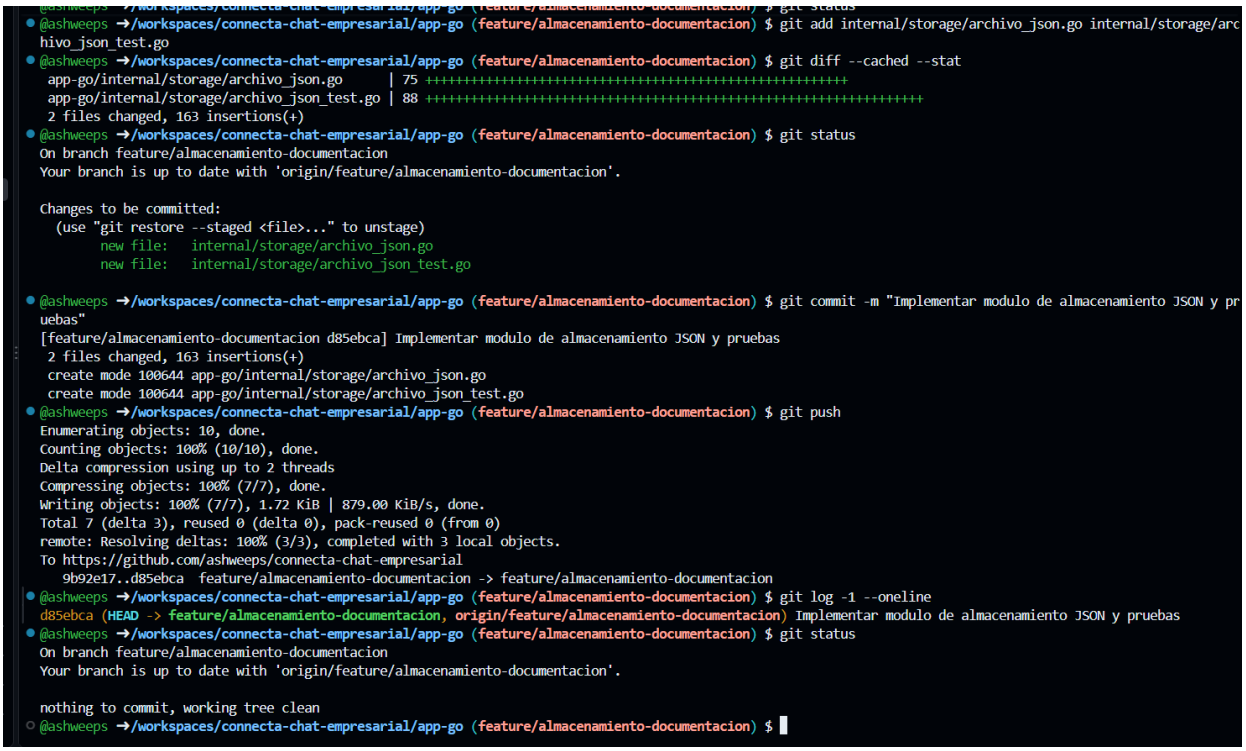
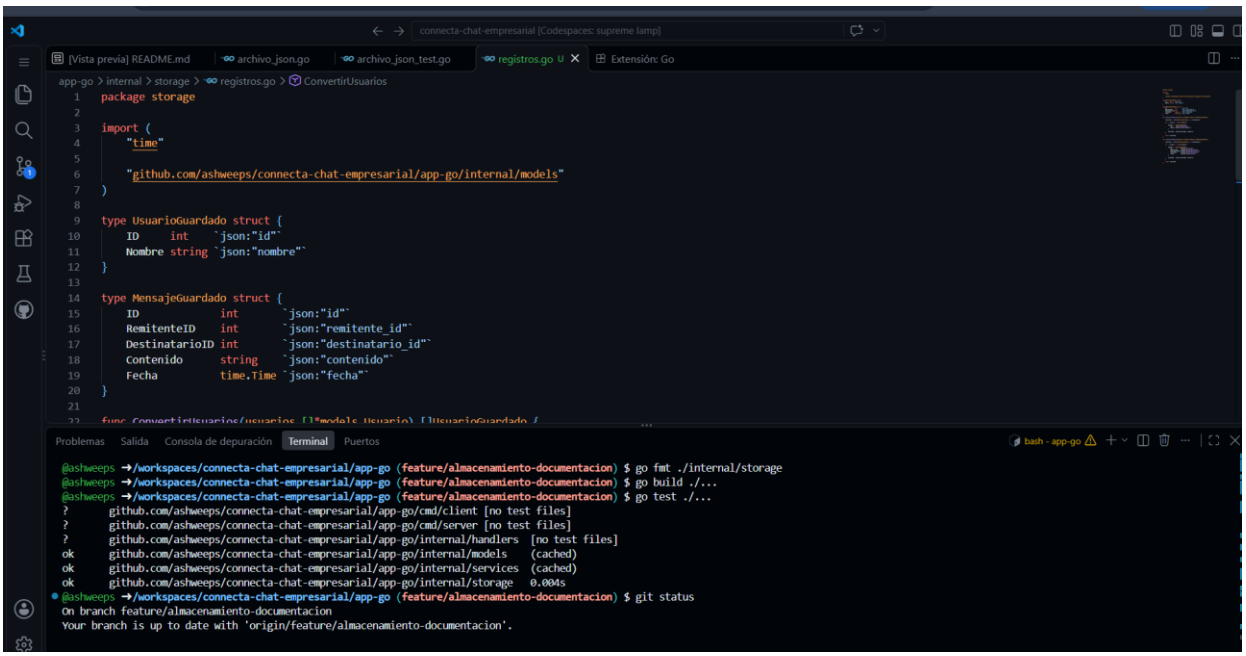

@ashweeps →/workspaces/connecta-chat-empresarial/app-go (feature/almacenamiento-documentacion) $ git add internal/storage/archivo_json.go internal/storage/archivo_json_test.go
@ashweeps →/workspaces/connecta-chat-empresarial/app-go (feature/almacenamiento-documentacion) $ git diff --cached --stat
app-go/internal/storage/archivo_json.go | 75 +++++++

Figura 9

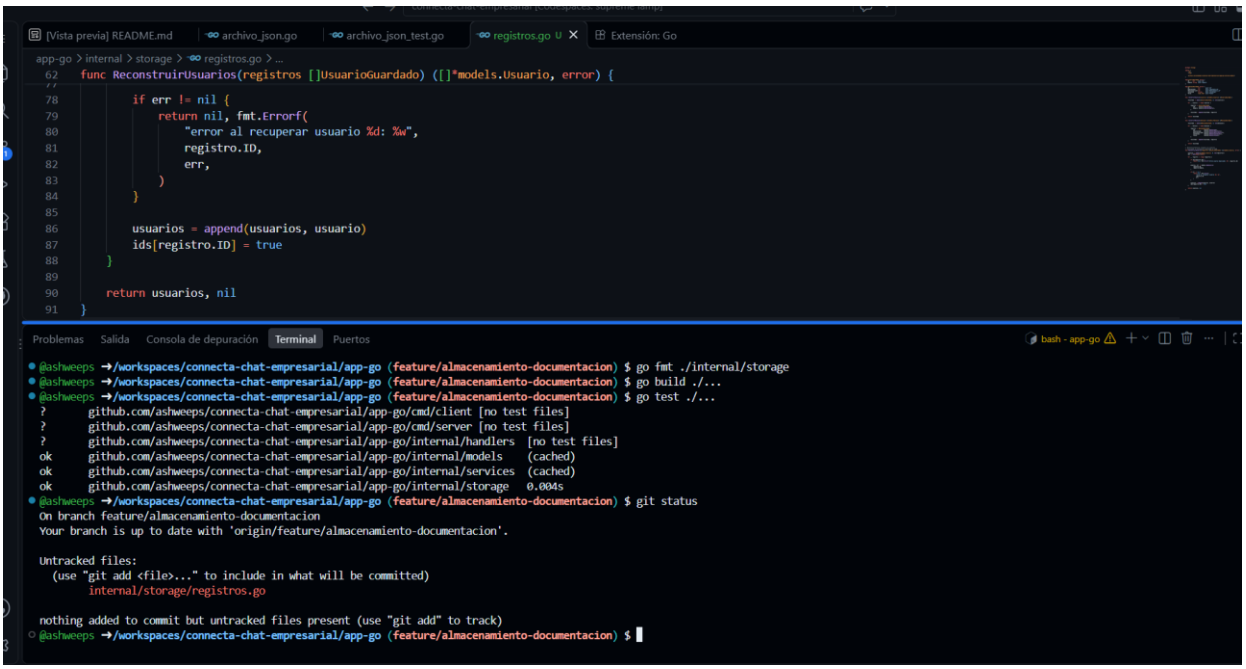
Estructuras para almacenar usuarios y mensajes



Nota. Se observa registros.go con las estructuras UsuarioGuardado y MensajeGuardado, junto con la compilación y pruebas correctas. Estas estructuras definen los campos que se guardarán en JSON. Captura de elaboración propia.

Figura 10

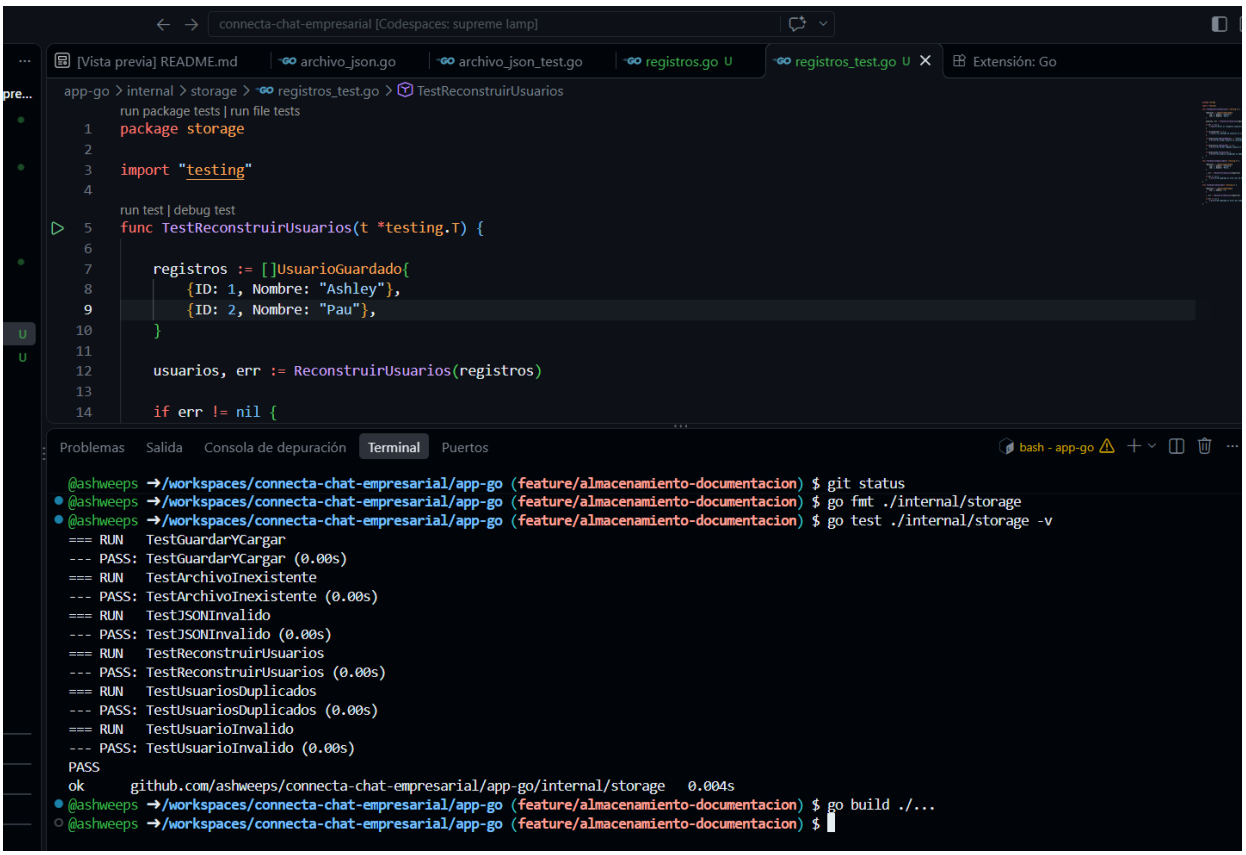
Función de reconstrucción de usuarios desde registros



Nota. La captura muestra parte de ReconstruirUsuarios y los resultados de compilación y pruebas. La función reutiliza el constructor del modelo para recuperar identificadores y nombres. Captura de elaboración propia.

Figura 11

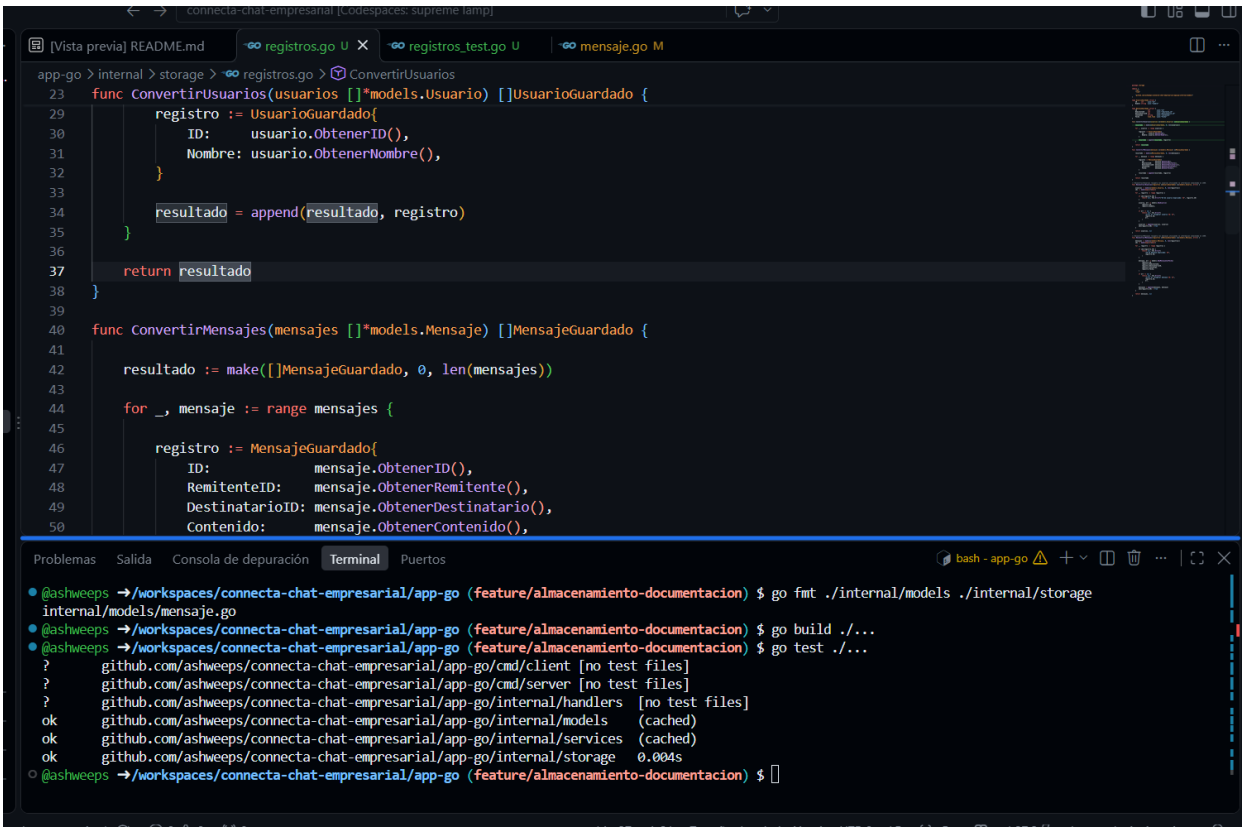
Validación de la reconstrucción de usuarios



Nota. Se observan resultados aprobados para reconstruir usuarios, identificar duplicados y rechazar nombres vacíos. Las pruebas respaldan la validación de datos antes de recuperar registros. Captura de elaboración propia.

Figura 12

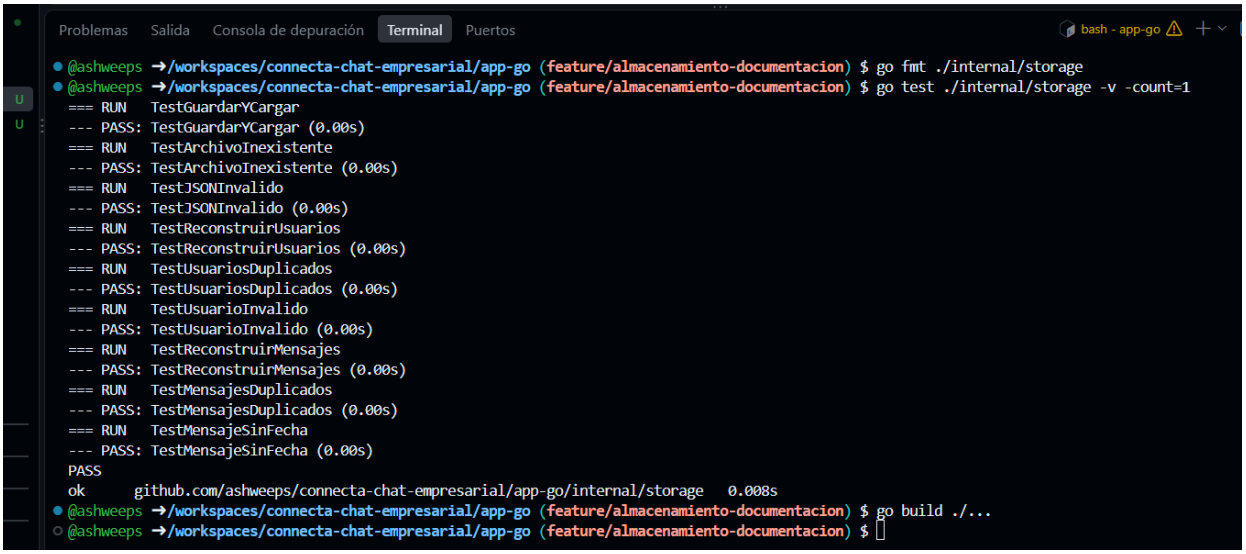
Preparación de mensajes conservando su fecha original



Nota. La terminal muestra la compilación después de incorporar el constructor de recuperación con fecha. El código prepara los mensajes para recuperar sus datos originales, aunque en esta fase aún faltaba una prueba específica de fechas. Captura de elaboración propia.

Figura 13

Pruebas de recuperación de mensajes y fechas



Nota. Se observan nueve pruebas aprobadas en el paquete storage. Entre ellas se comprueba la fecha original, la identificación de mensajes duplicados y el rechazo de registros sin fecha. Captura de elaboración propia.

Figura 14

Revisión de los archivos incorporados al segundo commit

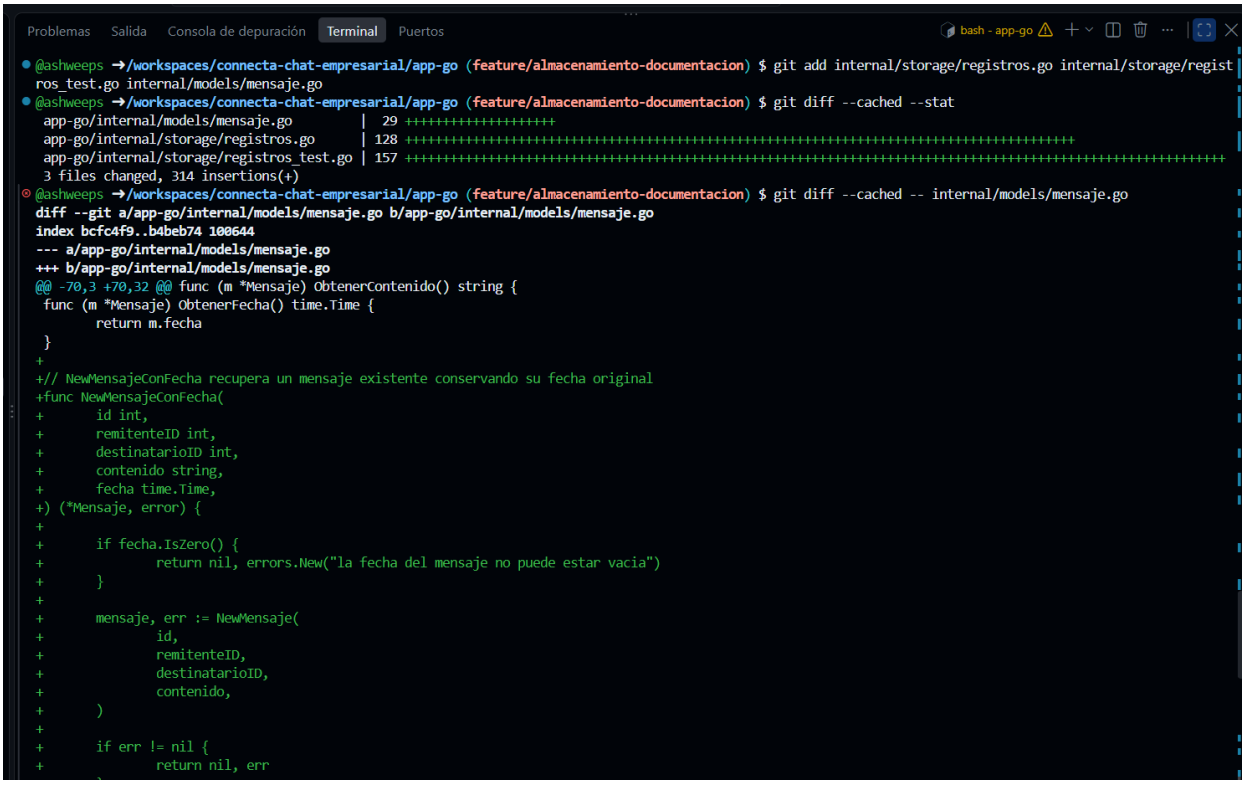
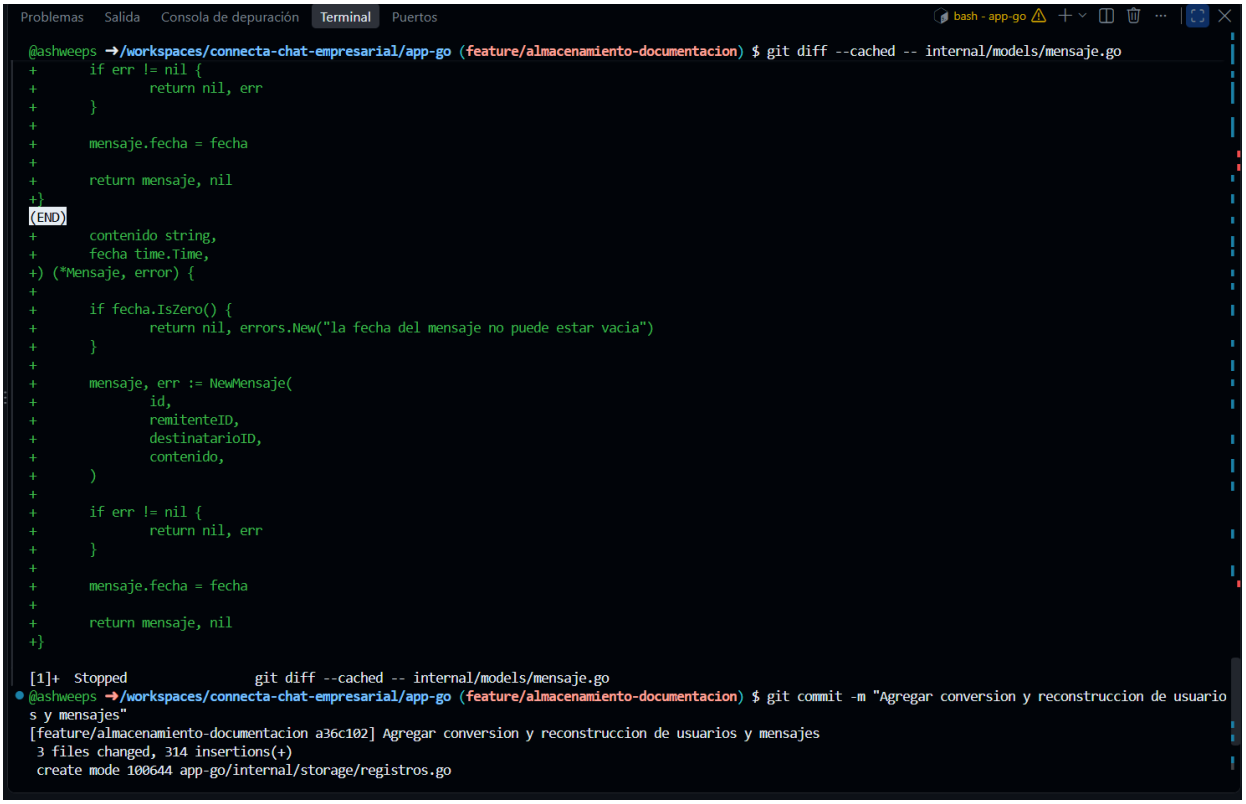

Problemas
Salida
Consola de depuración
Terminal
Puertos
bash - app-go
@ashweeps →/workspaces/connecta-chat-empresarial/app-go (feature/almacenamiento-documentacion) $ git add internal/storage/registros.go internal/storage/registros_test.go internal/models/mensaje.go
@ashweeps →/workspaces/connecta-chat-empresarial/app-go (feature/almacenamiento-documentacion) $ git diff --cached --stat
app-go/internal/models/mensaje.go | 29 ++++++++++++++++++++
app-go/internal/storage/registros.go | 128 ++++++++++++++++++++

Figura 15

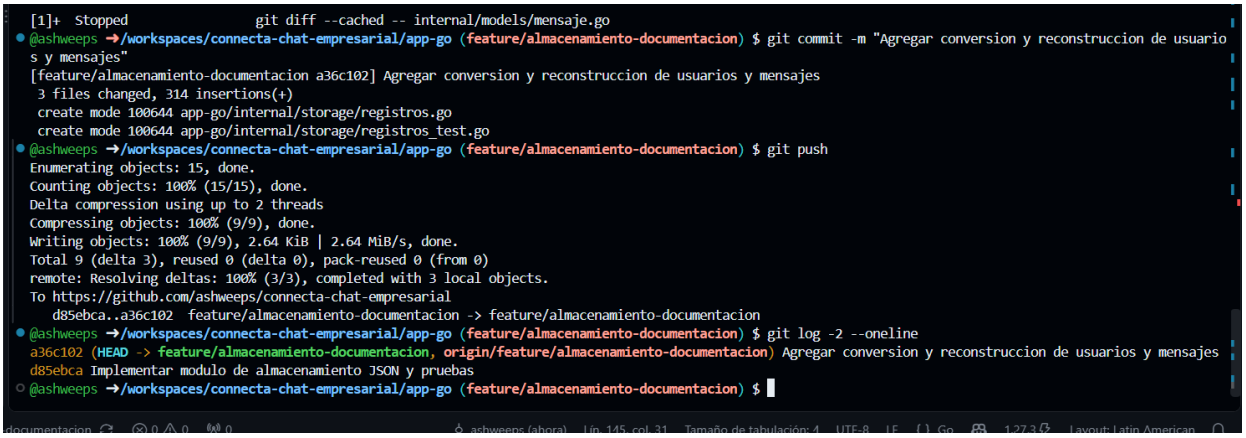
Revisión de un constructor del modelo y segundo commit



Nota. Se observa la incorporación de NewMensajeConFecha al modelo Mensaje y la ejecución del commit a36c102. Este constructor permite recuperar registros sin perder la fecha de envío original. Captura de elaboración propia.

Figura 16

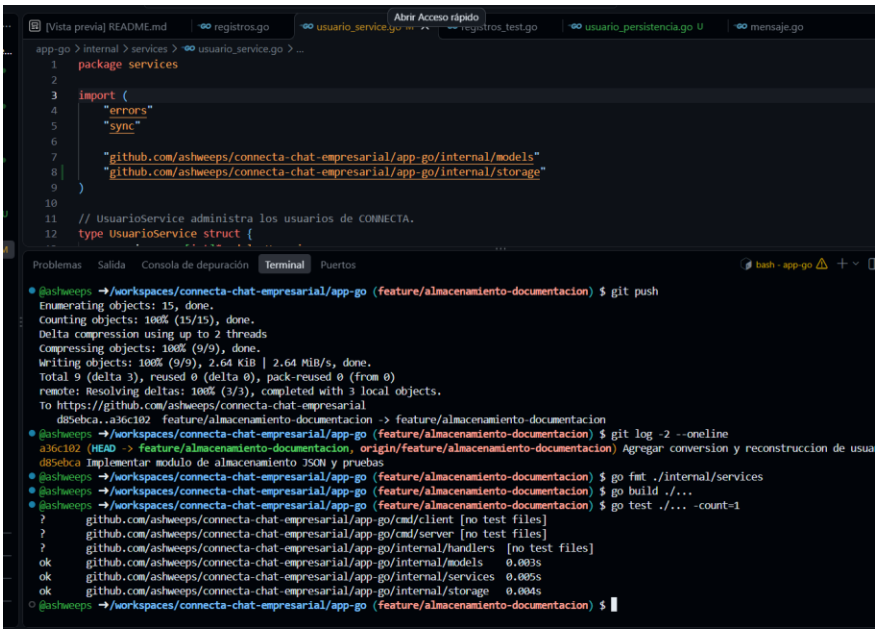
Publicación del segundo commit de almacenamiento



Nota. La terminal confirma el envío del commit a36c102 a GitHub y muestra el historial de los dos primeros commits. Se identifica la trazabilidad de la conversión y reconstrucción de registros. Captura de elaboración propia.

Figura 17

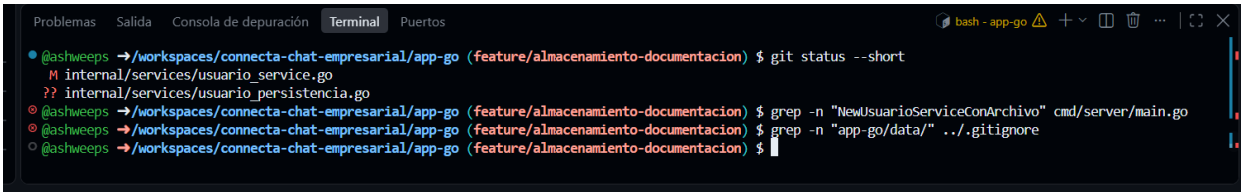
Compilación tras preparar la persistencia de usuarios



Nota. Se muestran el archivo usuario_service.go y los comandos go build y go test ejecutados sin errores. Esta captura acredita compatibilidad inicial, antes de la prueba real de almacenamiento. Captura de elaboración propia.

Figura 18

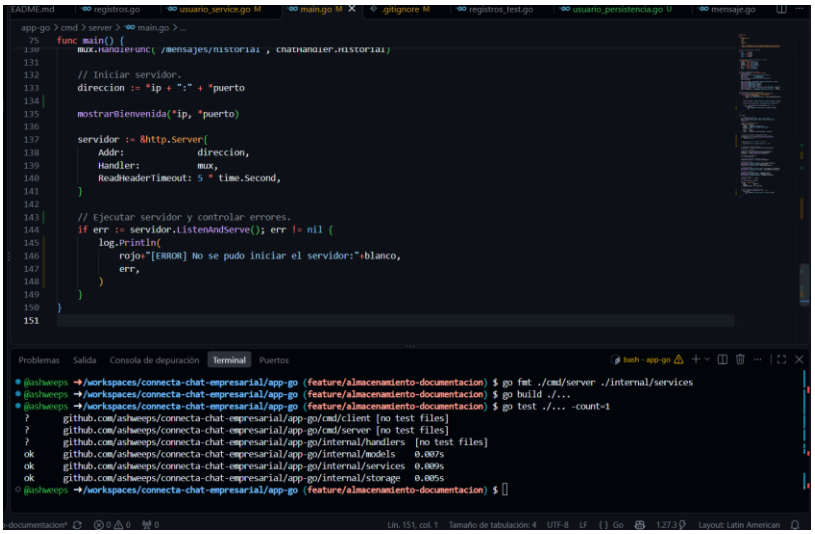
Detección de configuraciones pendientes de integración



Nota. La salida de git status identifica archivos pendientes. Las búsquedas no encontraron todavía el constructor de almacenamiento en main.go ni la exclusión de data en .gitignore, lo que orientó las correcciones posteriores. Captura de elaboración propia.

Figura 19

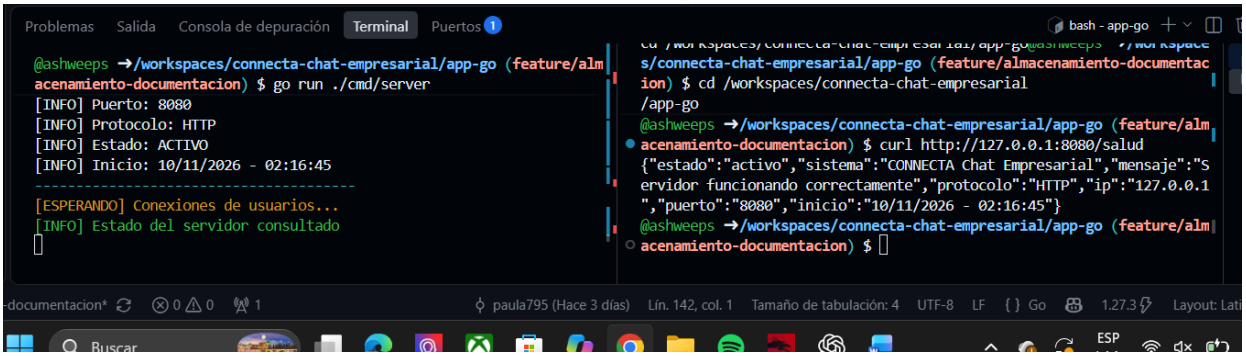
Compilación del servidor con persistencia de usuarios



Nota. Se observa el archivo principal del servidor y la ejecución correcta de formato, compilación y pruebas. El resultado comprueba que el proyecto seguía compilando al conectar el almacenamiento de usuarios. Captura de elaboración propia.

Figura 20

Respuesta HTTP de la ruta de estado del servidor



Nota. La captura muestra CONNECTA iniciado en 127.0.0.1:8080 y la respuesta JSON a una consulta curl /salud. La prueba demuestra disponibilidad del servidor, no conservación de datos. Captura de elaboración propia.

Figura 21

Consulta inicial de usuarios desde HTTP



Nota. En la terminal se consulta el recurso /usuarios y el servidor devuelve una lista vacía. Este resultado permite establecer el estado previo al registro de las usuarias de prueba. Captura de elaboración propia.

Figura 22

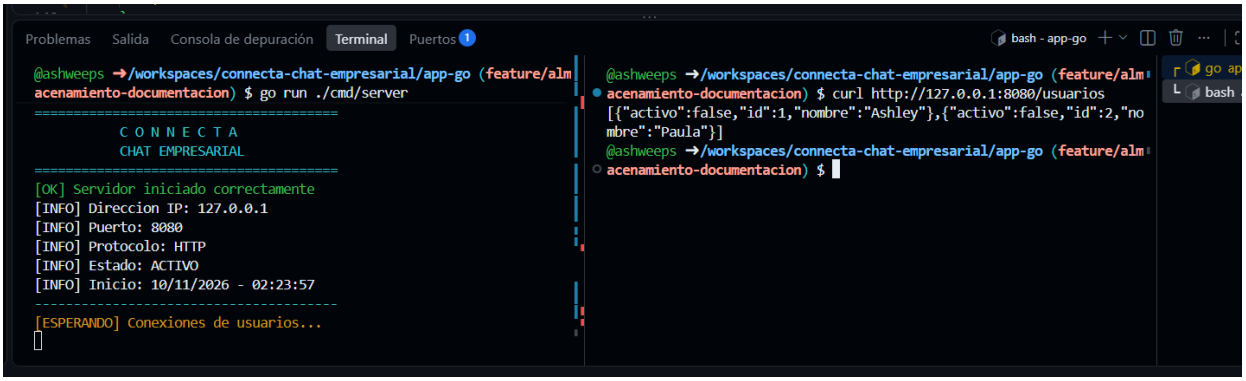
Registro de usuarias y escritura en usuarios.json



Nota. Se observa la creación de Ashley y Paula mediante solicitudes HTTP y el contenido real de data/usuarios.json. Los identificadores 1 y 2 y los nombres aparecen almacenados en el archivo. Captura de elaboración propia.

Figura 23

Recuperación de usuarios después de reiniciar el servidor



Nota. Después de reiniciar CONNECTA, la consulta HTTP devuelve los registros de Ashley y Paula con los identificadores originales y las sesiones inactivas. Esta evidencia confirma persistencia de usuarios. Captura de elaboración propia.

Figura 24

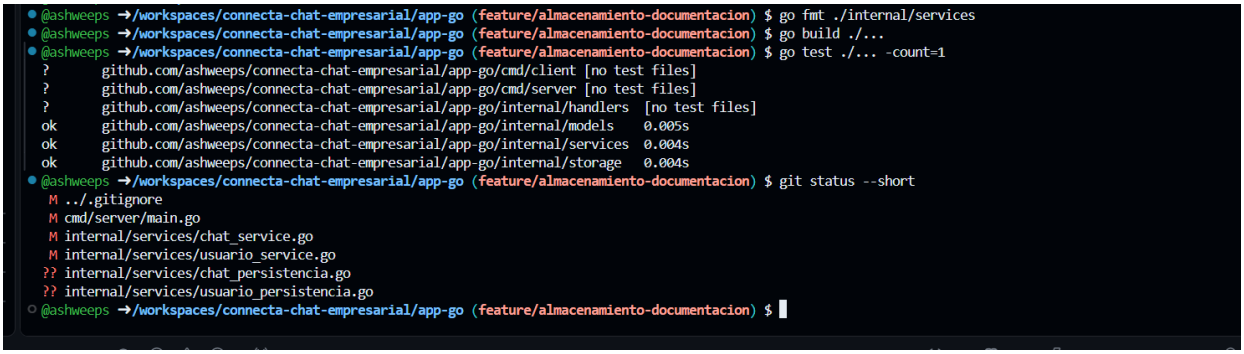
Continuidad de identificadores después del reinicio



Nota. La captura muestra un tercer registro, Camila, con ID 3, y el contenido actualizado de usuarios.json. Esta prueba comprueba que el contador de usuarios continúa sin repetir los identificadores anteriores. Captura de elaboración propia.

Figura 25

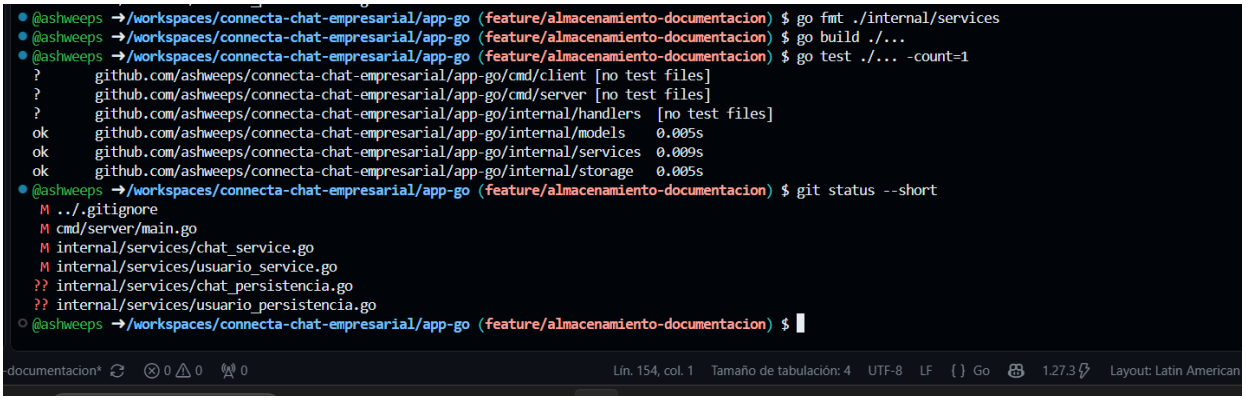
Preparación del servicio persistente de mensajes



Nota. Se observan la compilación, las pruebas correctas y los archivos chat_service.go y chat_persistencia.go entre los cambios. La captura corresponde a la preparación del módulo, antes de comprobar el guardado real. Captura de elaboración propia.

Figura 26

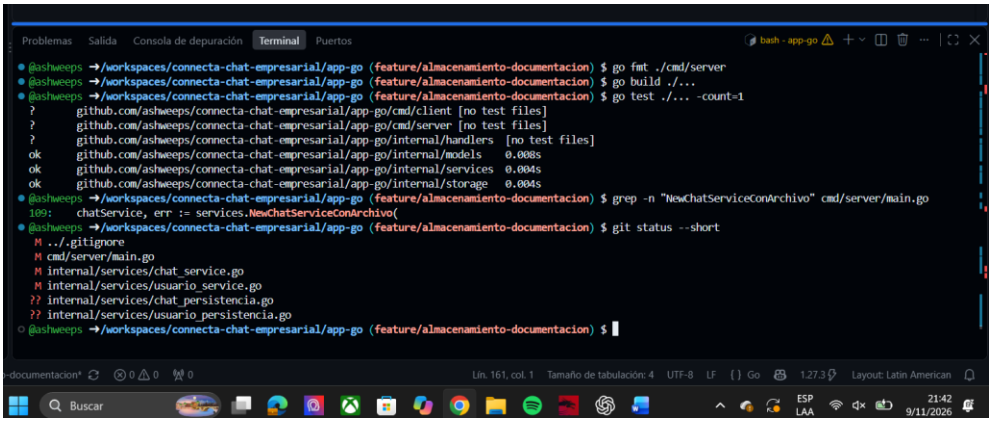
Validación del guardado JSON desde el servicio de chat



Nota. Se muestran go build y go test finalizados correctamente después de incorporar guardarMensajesBloqueados al envío de mensajes. Esta evidencia confirma compatibilidad, no recuperación tras reinicio. Captura de elaboración propia.

Figura 27

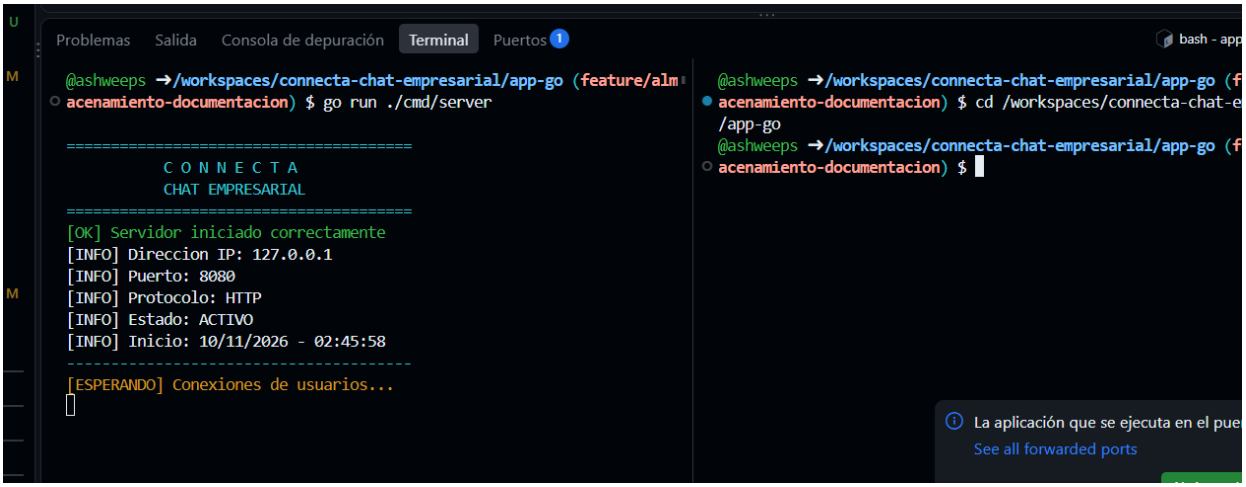
Conexión del servicio de chat persistente con el servidor



Nota. La búsqueda en main.go confirma la llamada NewChatServiceConArchivo y las pruebas continúan sin errores. La captura respalda la conexión del código antes de la prueba operativa. Captura de elaboración propia.

Figura 28

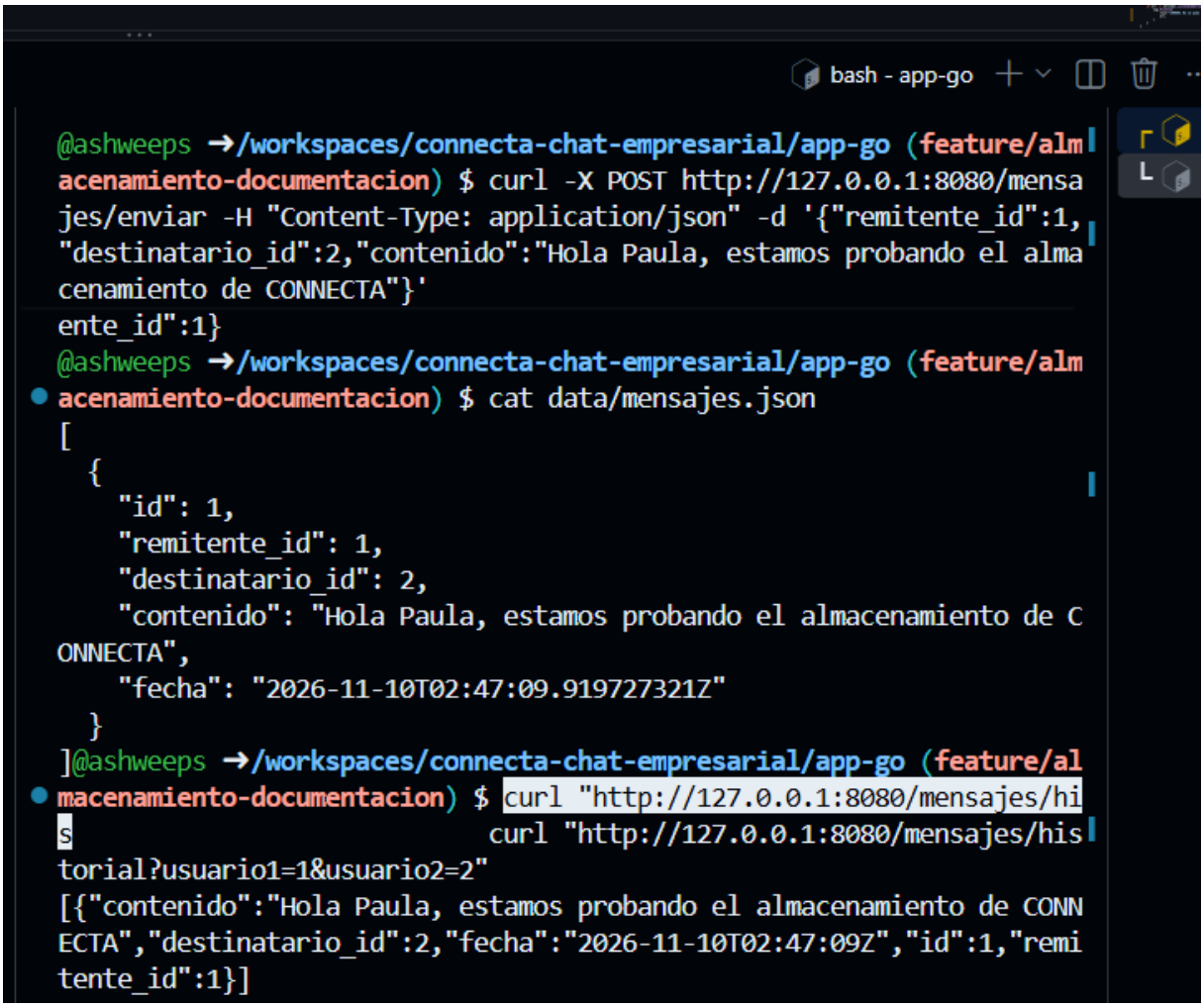
Inicio del servidor actualizado para probar mensajería



Nota. Se observa el servidor activo en el puerto 8080 y una segunda terminal lista para realizar consultas. Este paso preparó la prueba HTTP de almacenamiento de mensajes. Captura de elaboración propia.

Figura 29

Envío de un mensaje y verificación de mensajes.json



Nota. La terminal muestra una solicitud POST para enviar un mensaje de Ashley a Paula, el archivo mensajes.json con el registro y una consulta del historial. El contenido y la fecha coinciden con el mensaje enviado. Captura de elaboración propia.

Figura 30

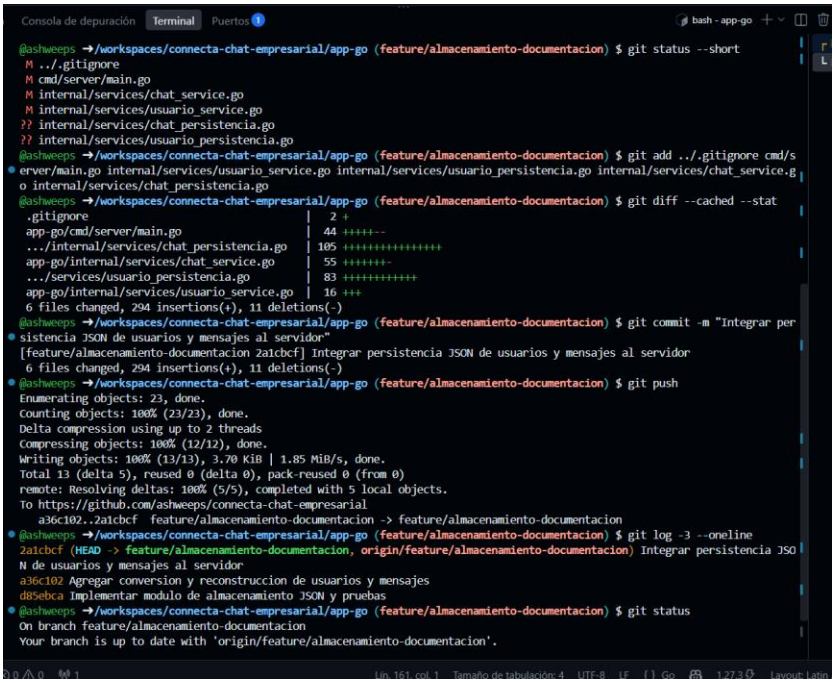
Recuperación del historial después de reiniciar el servidor



Nota. Tras un nuevo inicio del servidor, la consulta del historial devuelve el mensaje anterior con ID 1 y su fecha original. Esta evidencia confirma la persistencia y recuperación de mensajes. Captura de elaboración propia.

Figura 31

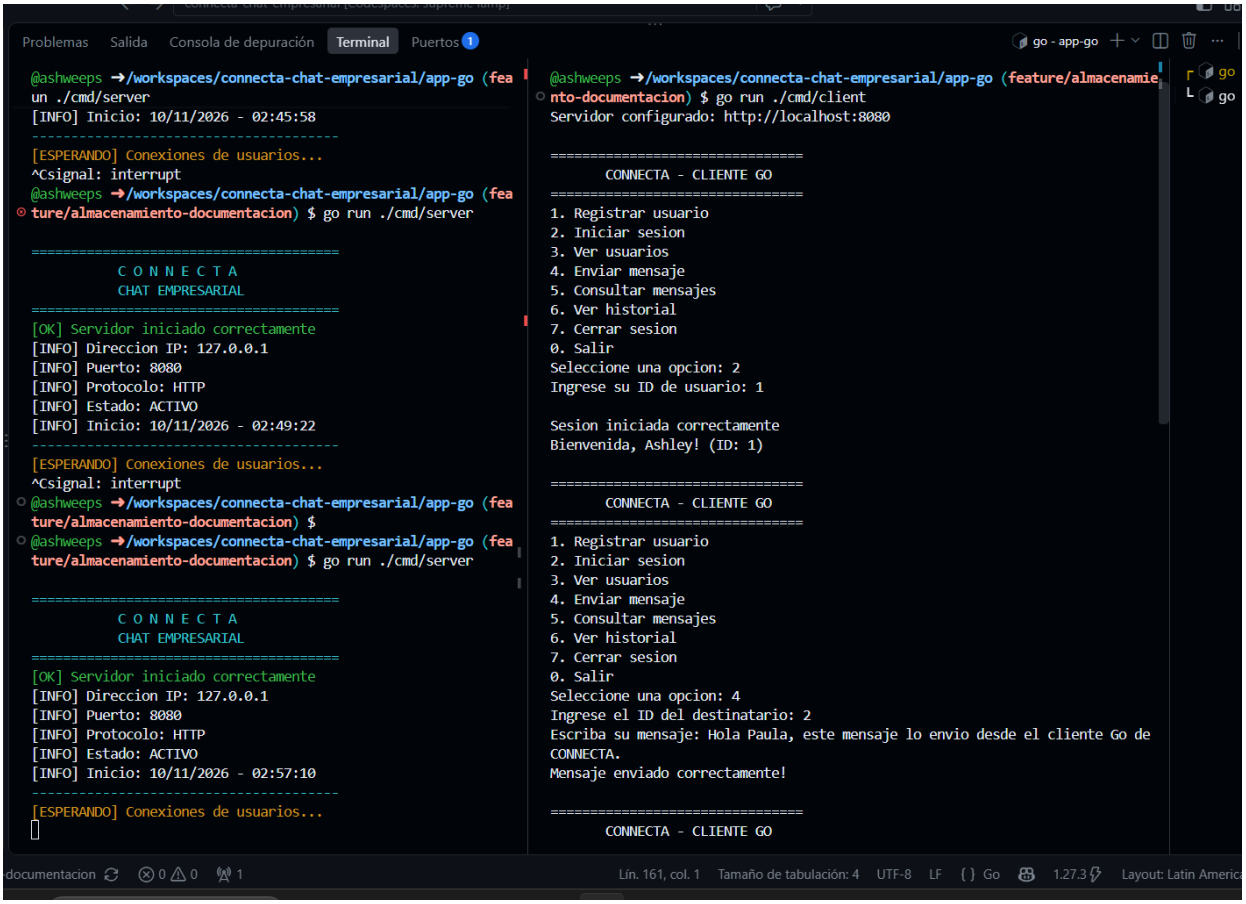
Tercer commit de integración y publicación en GitHub



Nota. Se muestran seis archivos preparados y el commit 2a1cbcf, publicado mediante git push en la rama de almacenamiento. El historial registra las tres contribuciones de programación de la Persona 3. Captura de elaboración propia.

Figura 32

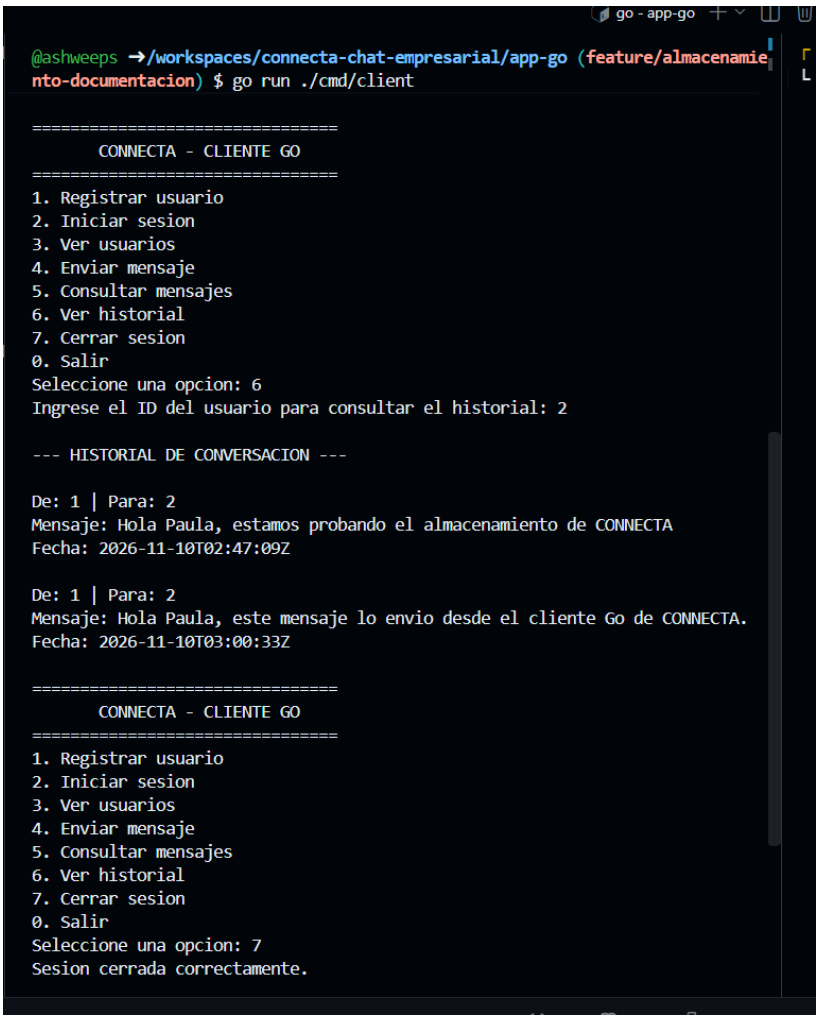
Inicio de sesión y envío desde el cliente Go



Nota. La imagen presenta el servidor en ejecución y el cliente Go con la sesión de Ashley (ID 1) iniciada. El cliente confirma el envío de un mensaje a Paula (ID 2), lo que acredita la integración entre cliente y servidor. Captura de elaboración propia.

Figura 33

Consulta de historial y cierre de sesión desde el cliente



Nota. La captura muestra dos mensajes recuperados entre los usuarios 1 y 2, con sus fechas, y la confirmación del cierre de sesión. Se verifica la consulta del historial desde el cliente real. Captura de elaboración propia.

Link del video

https://drive.google.com/file/d/1sKFkosUU5ZP26_I583_l2wZUEM4krBM6/view?usp=drivesdk

Link del repo

https://github.com/ashweeps/connecta-chat-empresarial

Referencias

Referencias

American Psychological Association. (s. f.). *Student paper checklist (Publication Manual): APA Style, 7th edition*. https://apastyle.apa.org/instructional-aids/publication-manual-formatting-checklist.pdf

Equipo CONNECTA. (2026). *CONNECTA: Chat empresarial* [Código fuente]. GitHub. https://github.com/ashweeps/connecta-chat-empresarial

Purdue Online Writing Lab. (s. f.). *APA tables and figures*. Purdue University. https://owl.purdue.edu/owl/research_and_citation/apa_style/apa_formatting_and_style_guide/apa_tables_and_figures.html

The Go Authors. (s. f.-a). *A Tour of Go*. https://go.dev/tour/

The Go Authors. (s. f.-b). *Effective Go*. https://go.dev/doc/effective_go

The Go Authors. (s. f.-c). *Package encoding/json*. https://pkg.go.dev/encoding/json

The Go Authors. (s. f.-d). *Package errors*. https://pkg.go.dev/errors

The Go Authors. (s. f.-e). *Package fmt*. https://pkg.go.dev/fmt

The Go Authors. (s. f.-f). *Package net/http*. https://pkg.go.dev/net/http

The Go Authors. (s. f.-g). *Package os*. https://pkg.go.dev/os

The Go Authors. (s. f.-h). *Package path/filepath*. https://pkg.go.dev/path/filepath

The Go Authors. (s. f.-i). *Package reflect*. https://pkg.go.dev/reflect

The Go Authors. (s. f.-j). *Package sort*. https://pkg.go.dev/sort

The Go Authors. (s. f.-k). *Package sync*. https://pkg.go.dev/sync

The Go Authors. (s. f.-l). *Package testing*. https://pkg.go.dev/testing

The Go Authors. (s. f.-m). *Package time*. https://pkg.go.dev/time